Flaix ↔ BreakEat — les matchs, les commandes et la Live Activity

À l'attention du développeur Flaix · BreakEat · 11 novembre 2026 · version 1

Le but. Flaix devient **la seule saisie**. Les matchs créés dans Flaix apparaissent tout seuls dans l'app BreakEat, et les commandes passées dans Flaix y remontent avec ce que l'app sait déjà faire : « Mes commandes », la Live Activity sur l'écran verrouillé, le reçu avec la TVA, la note et la fidélité — et la notification « ta commande est prête », que BreakEat ajoute de son côté.

	Ce qu'on te demande	Pourquoi	Priorité
1	**Le lien calendrier** de chaque lieu	Le client ouvre ta page depuis l'app	Tout de suite
2	**Le flux des matchs** (JSON)	Plus de double saisie : les matchs arrivent tout seuls	Haute
3	**Les événements de commande** (webhook signé)	« Mes commandes », Live Activity, « prête », reçu, note, fidélité, campagnes	**La plus haute**
—	Le fichier iCal	Les supporters ajoutent les matchs à leur agenda	Facultatif

Rien n'est encore développé, ni chez toi ni chez nous. Ce dossier fixe le format pour que chacun avance de son côté. **BreakEat reçoit déjà des événements signés de Flaix pour la Live Activity** : la partie 3 prolonge ce qui existe au lieu de le remplacer.

1. Le lien calendrier de chaque lieu

1. **Une adresse par lieu**, stable dans le temps (elle sera enregistrée une fois pour toutes dans l'admin BreakEat).
2. La page s'affiche correctement **sur mobile, à l'intérieur d'une autre app** (« webview ») :
   - pas d'en-tête qui interdit l'intégration (`X-Frame-Options` , `Content-Security-Policy: frame-ancestors`) ;
   - la page **et le paiement** restent si possible sur `flaixlabs.com` ou `synertic.net` . Un autre domaine fonctionne (en HTTPS), mais l'app affiche alors au client un bandeau « Paiement sur ». **Nous dire quel prestataire de paiement est utilisé**, et sur quel domaine.
3. Chaque match mène à **sa** page de commande.

En plus : le lien doit accepter nos **étiquettes d'identité**, que BreakEat remplace par les coordonnées du client connecté avant d'ouvrir ta page — `[#firstname]` , `[#lastname]` , `[#mail]` , `[#Votre portable]` — et une **nouvelle** : `[#breakeatId]` , l'identifiant du client chez BreakEat. Garde-la avec la commande : c'est elle qui permet de la rattacher au bon client (partie 3).

2. Le flux des matchs

Une adresse par lieu, en lecture publique, qui renvoie en JSON la liste des matchs **à venir et des 7 derniers jours**.

Exemple d'adresse (le chemin exact est libre) :

```
GET https://<domaine-flaix>/api/lieux/<identifiant-du-lieu>/evenements
```

Exemple de réponse :

```
{
  "lieu": { "id": "velodrome", "nom": "Orange Vélodrome" },
  "genereLe": "2026-11-10T09:00:00Z",
  "evenements": [
    {
      "id": "m-123",
      "nom": "Marseille vs Lens",
      "debut": "2026-11-22T21:00:00+01:00",
      "fin": "2026-11-22T23:00:00+01:00",
      "adversaire": "RC Lens",
      "logoAdversaire": "https://<domaine-flaix>/logos/lens.png",
      "affiche": null,
      "ouvertureCommande": "2026-11-22T08:00:00+01:00",
      "lienCommande": "https://<domaine-flaix>/commande/m-123",
      "statut": "actif",
      "modifieLe": "2026-11-09T18:00:00Z"
    }
  ]
}
```

Les champs

Champ	Obligatoire	Exemple	Pourquoi
`id`	**oui**	`m-123`	Identifiant **stable** : BreakEat reconnaît un match modifié au lieu de le créer en double. **Ne jamais le réutiliser** pour un autre match.
`nom`	**oui**	`Marseille vs Lens`	Le titre affiché.
`debut` , `fin`	**oui**	`2026-11-22T21:00:00+01:00`	Format ISO 8601 **avec le fuseau horaire** (`+01:00` ou `Z`). Sans fuseau, on risque un décalage d'une heure au changement d'heure.
`adversaire`	non	`RC Lens`	Affichage du match.
`logoAdversaire`	non	`https://…/lens.png`	Image en HTTPS, carrée de préférence.
`affiche`	non	`https://…/affiche.jpg`	Pour les concerts et événements sans adversaire.
`ouvertureCommande`	**oui**	`2026-11-22T08:00:00+01:00`	L'heure à laquelle **la commande ouvre**. BreakEat affiche « La commande ouvre à 08:00 » en attendant. `null` = ouverte dès maintenant.

Champ	Obligatoire	Exemple	Pourquoi
`lienCommande`	**oui**	`https://…/commande/m-123`	Le bouton « Commander ». Peut accepter des paramètres (identité du client), comme les liens actuels.
`statut`	**oui**	`actif` ou `annule`	Un match annulé doit disparaître partout. **Ne pas le retirer de la liste** : le passer en `annule`, sinon BreakEat ne peut pas faire la différence entre « annulé » et « oublié ».
`modifieLe`	**oui**	`2026-11-09T18:00:00Z`	Date de la dernière modification du match.

Les règles

- Matchs **triés par date de début**.
- Adresse **sans mot de passe**, en **lecture seule**. Elle ne doit contenir **aucune donnée client** (ni nom, ni commande) : seulement le calendrier public.
- BreakEat l'interrogera **toutes les 5 à 10 minutes** par lieu, pour une cinquantaine de lieux : une réponse **rapide** (moins d'une seconde), mise en cache côté Flaix si besoin. Un en-tête `ETag` ou `Last-Modified` est un plus.
- Réponse `200` avec une liste vide s'il n'y a aucun match ; `404` seulement si le lieu n'existe pas.
- Encodage UTF-8 (accents des noms de clubs).

En option : le fichier iCal

La même liste au format `.ics`, une adresse par lieu :

```
GET https://<domaine-flaix>/api/lieux/<identifiant-du-lieu>/evenements.ics
```

Correspondance : `UID` = `id`, `SUMMARY` = `nom`, `DTSTART` / `DTEND` = `debut` / `fin`, `URL` = `lienCommande`, `STATUS:CANCELLED` pour un match annulé, `LAST-MODIFIED` = `modifieLe`. L'heure d'ouverture de la commande peut aller dans un champ `X-FLAIX-OUVERTURE-COMMANDE`. Généralement une demi-journée de travail une fois B fait.

3. Les commandes et la Live Activity

C'est la partie la plus importante. Aujourd'hui, **une commande passée dans Flaix ne crée rien chez BreakEat**. Tout ce que l'app sait déjà faire avec une commande en dépend :

Ce que le client voit dans l'app	Ce que BreakEat doit recevoir
« Mes commandes » : le ticket, la buvette, « Commande n° 142 », le prix, la date	la commande, sa buvette, son numéro, son total, sa date
La barre « reçue → en préparation → prête »	chaque changement d'état, **en temps réel**
La **Live Activity** (écran verrouillé, Dynamic Island) et son décompte	l'état, et l'**heure estimée** à laquelle elle sera prête

Ce que le client voit dans l'app	Ce que BreakEat doit recevoir
« Ta commande est prête » (notification — ajoutée côté BreakEat)	l'état `READY`, dans les secondes qui suivent
L'heure de retrait sur le ticket	le créneau choisi, ou « dès que prête »
Le **reçu** avec la TVA taux par taux	chaque ligne : produit, quantité, prix TTC, **taux de TVA**
Le bandeau « produit manquant »	les lignes qui manquent, et combien
Le bandeau de remboursement	le montant remboursé
« Noter ma commande », les points de fidélité	l'état `COLLECTED` (récupérée)
Les campagnes vers « ceux qui ont commandé »	**qui** a commandé : l'identifiant BreakEat du client

Ce qui existe déjà et qu'on garde. BreakEat reçoit DÉJÀ des événements signés de Flaix pour la Live Activity : même signature HMAC-SHA256, même anti-rejeu (5 minutes), même idempotence par `eventId`, mêmes noms d'événements (`ORDER_STATUS_CHANGED`, `ORDER_READY`, `ORDER_DELAYED`, `ORDER_CANCELLED`, `ORDER_COLLECTED`, `PICKUP_SLOT_CHANGED`, `PICKUP_POINT_CHANGED`). Seule limite : ils désignent une commande **BreakEat** (`orderId`), qui n'existe pas quand on commande chez Flaix. D'où deux ajouts seulement :

1. **Un événement** `ORDER_CREATED` qui porte toute la commande — la première fois que BreakEat en entend parler ;
2. `flaixOrderId` à la place de `orderId` dans tous les événements : la référence Flaix, stable. BreakEat fait la correspondance.

```
POST https://<api-breakeat>/api/v1/webhooks/flaix
X-Flaix-Signature: <HMAC-SHA256 du corps brut, secret partagé>
{
  "eventId": "evt-5521",                 // unique : clé d'idempotence
  "event": "ORDER_CREATED",
  "timestamp": "2026-11-22T20:12:04Z",   // anti-rejeu
  "flaixOrderId": "c-98765",             // stable, jamais réutilisé
  "lieu": "velodrome",                   // id Flaix du lieu
  "evenement": "m-123",                  // id du match (le même que dans B)
  "client": {
    "breakeatId": "4b1e…",               // rendu par l'étiquette [#breakeatId]
    "email": "client@exemple.fr"         // de secours, pour le rapprochement
  },
  "numero": "142",                       // ce que le client dit au comptoir
  "buvette": { "id": "b-nord", "nom": "Tribune Nord" },
  "retrait": {
    "creneau": "2026-11-22T21:45:00+01:00",   // null = « dès que prête »
    "point": "Comptoir 3"
  },
  "lignes": [
    { "produit": "Hot-dog", "quantite": 2, "totalCentimes": 1200, "tvaPourDixMille": 1000 },
    { "produit": "Coca 50 cl", "quantite": 1, "totalCentimes": 450, "tvaPourDixMille": 550 }
  ],
  "totalCentimes": 1650,
  "payeeLe": "2026-11-22T20:12:01+01:00",
  "status": "CREATED",
  "estimatedReadyAt": "2026-11-22T20:25:00+01:00",
  "lienRecu": null                       // facultatif : le reçu officiel Flaix
}
```

Puis, à chaque étape, un événement court :

```
{ "eventId": "evt-5530", "event": "ORDER_READY", "timestamp": "…",
  "flaixOrderId": "c-98765", "status": "READY", "estimatedReadyAt": null }
```

Deux événements de plus, pour ce que l'app affiche déjà :

- **`ORDER_ITEMS_MISSING`** — `{ "lignes": [{ "produit": "Hot-dog", "manquant": 1 }] }` : le bandeau « produit manquant » ;
- **`ORDER_REFUNDED`** — `{ "rembourseCentimes": 600, "total": false }` : le bandeau de remboursement (`total: true` = remboursée entièrement).

Les exigences :

- **Le client est reconnu** grâce à une nouvelle étiquette dans le lien de commande, `[#breakeatId]`, que BreakEat remplit déjà pour `[#firstname]`, `[#mail]`, etc. Flaix la garde avec la commande et la renvoie ici. Sans elle, la commande ne peut être rattachée à personne : ni « Mes commandes », ni Live Activity.
- **Temps réel** : `ORDER_READY` part **dans les secondes** qui suivent le geste du comptoir — c'est lui qui fait sonner le téléphone.
- **Dans l'ordre, sans doublon** : un événement plus ancien qu'un autre déjà reçu est ignoré ; un événement rejoué (même `eventId`) aussi.
- **Réessais** en cas d'échec (par exemple 10 s, 1 min, 5 min, 30 min) : une coupure de quelques minutes ne doit perdre aucune commande.

- **Une adresse de relecture** : `GET /api/commandes/<flaixOrderId>` côté Flaix (avec une clé d'API fournie par Flaix), pour que BreakEat rattrape une commande dont il a manqué un événement.
- **Le taux de TVA par ligne** est indispensable : sans lui, le reçu n'est plus un justificatif. Si Flaix émet déjà son propre reçu, `lienRecu` permet à l'app de l'afficher à la place du sien.

Dans l'autre sens (BreakEat → Flaix), un seul signal : « **Je suis arrivé** ». Le client le touche quand il est devant le comptoir ; aujourd'hui il n'arrive qu'à BreakEat. Pour que le comptoir Flaix le voie : `POST https://<domaine-flaix>/api/commandes/<flaixOrderId>/arrivee` (avec une clé d'API fournie par Flaix), sans corps. À Flaix de dire s'il sait l'afficher au comptoir.

4. Sécurité

- **Deux secrets, jamais dans une app ni dans un navigateur** : le secret de signature des événements (partie 3), et, si tu l'utilises, la clé de l'API de relecture. Seuls les deux serveurs les connaissent.
- Ils s'échangent **par un canal sûr** (gestionnaire de mots de passe partagé, message chiffré) — **jamais par e-mail en clair**.
- **Révocables et remplaçables sans coupure** : deux secrets valides pendant le changement.
- Le flux des matchs (partie 2) est **public et en lecture seule** : il ne contient **aucune donnée client**.
- Une panne d'un côté **n'arrête jamais l'autre** : BreakEat garde les derniers matchs connus, Flaix réessaie ses envois.

5. Ce qu'on attend de toi

1. **Faisable, et pour quand ?** Partie par partie (1, 2, 3).
2. **L'étiquette** `[#breakeatId]` dans le lien de commande : acceptée ?
3. **La TVA par ligne** dans les événements de commande : disponible ? Et émets-tu déjà ton propre reçu (`lienRecu`) ?
4. **« Je suis arrivé »** : peux-tu l'afficher au comptoir ?
5. **Un environnement de test** : l'adresse du flux des matchs, l'adresse de relecture d'une commande, **un lieu de test** avec un match et des produits, et le **secret de signature** (par canal sûr).
6. Le **domaine** de la page de commande et celui du **paiement** (pour l'affichage dans l'app).

6. Ce que BreakEat fait de son côté

Pour information — rien à faire chez toi.

- **Les matchs** : un import automatique toutes les quelques minutes, qui crée ou met à jour les matchs par leur `id` Flaix et retire les matchs annulés. **1 à 2 jours** une fois une adresse de test disponible.
- **Les commandes** : la réception de `ORDER_CREATED` et des événements suivants, le rattachement au client par `[#breakeatId]`, puis « Mes commandes », la Live Activity, la notification « prête », le reçu, les bandeaux « produit manquant » et « remboursement », la note et la fidélité — **avec le design actuel de l'app**. **1 à 2 jours** une fois des événements de test reçus.
- **L'étiquette** `[#breakeatId]` dans les liens de commande, et l'envoi de « Je suis arrivé ».

Plus tard, si besoin (non demandé aujourd'hui) : ouvrir le tableau de bord Flaix depuis BreakEat sans se reconnecter (lien d'accès signé), et afficher dans le tableau de bord Flaix les chiffres de l'app (fréquentation, favoris, notifications) ou lancer une notification depuis Flaix. Ces sujets ont leur propre cahier des charges, disponible sur demande.